freeing the pointed-to memory or just changing an associated value.
6. A thread then changes x back to A based on this new data. If this is a pointer, it may be a new object that just happens to share the same address as the old one.
7. Thread 1 resumes and performs a compare/exchange on x, comparing against A. The compare/exchange succeeds (because the value is indeed A), but this is *the wrong* A *value*. The data originally read at step 2 is no longer valid, but thread 1 has no way of telling and will thus corrupt the data structure.

None of the algorithms presented here suffer from this problem, but it's easy to write lock-free algorithms that do. The most common way to avoid the problem is to include an ABA counter alongside the variable x. The compare/exchange operation is then done on the combined structure of x plus the counter as a single unit. Every time the value is replaced, the counter is incremented, so even if x has the same value, the compare/exchange will fail if another thread has modified x.

The ABA problem is particularly prevalent in algorithms that use free lists or otherwise recycle nodes rather than returning them to the allocator.

7.3.4. Guideline: identify busy-wait loops and help the other thread

In the final queue example you saw how a thread performing a push operation had to wait for another thread also performing a push to complete its operation before it could proceed. Left alone, this would have been a busy-wait loop, with the waiting thread wasting CPU time while failing to proceed. If you end up with a busy-wait loop, you effectively have a blocking operation and might as well use mutexes and locks. By modifying the algorithm so that the waiting thread performs the incomplete steps if it's scheduled to run before the original thread completes the operation, you can remove the busy-wait and the operation is no longer blocking. In the queue example this required changing a data member to be an atomic variable rather than a nonatomic variable and using compare/exchange operations to set it, but in more complex data structures it might require more extensive changes.